

Narzędzia i Aplikacje Jakarta EE

Jakarta Enterprise Beans

Michał Wójcik

Specyfikacja Jakarta Enterprise Beans

Jakarta Enterprise Beans - dawniej Enterprise Java Beans (EJB):

- beany zarządzane przez kontener serwera aplikacji,
- kontener udostępnia programiście dodatkowe usługi jak zarządzanie transakcjami, bezpieczeństwem, itd,
- potrafi równoważyć obciążenie na kilka serwerów aplikacji (skalowalność),
- klientem dla EJB może być wiele typów aplikacji (nie tylko moduł war),
- bardzo dobrze nadają się do dużych aplikacji rozproszonych,
- dzięki uproszczonemu tworzeniu i wdrażaniu EJB dobrze wspomagają tworzenie mniejszych aplikacji.

Rodzaje EJB

Rodzaje EJB:

- sesyjne - wykonują określone zadania dla klienta (mogą implementować usługi sieciowe):
  - bezstanowe,
  - stanowe,
  - singleton;
- sterowane komunikatami (message-driven) - dają możliwość "luźnego" łączenia komponentów aplikacji.

Interfejsy EJB

Sesyjne EJB mogą udostępniać dwa rodzaje interfejsów:

- lokalny:
  - może być spakowany w module EJB, jak i w module webowym,
  - klienty muszą działać w tej samej maszynie wirtualnej co serwer,
  - mogą z nich korzystać aplikacje webowe i inne EJB,
  - ich położenie nie jest przezroczyste dla klienta;
- zdalny:
  - muszą być spakowane w module EJB,
  - klienty mogą działać w innej maszynie wirtualnej niż EJB,
  - mogą z nich korzystać aplikacje webowe, inne EJB, oddzielne aplikacje klienckie,
  - ich położenie jest przezroczyste dla klienta.

Interfejsy EJB – różnice

Pisząc aplikację wykorzystującą interfejsy zdalne, należy pamiętać o kilku ważnych aspektach:

- argumenty i rezultaty metod EJB muszą być serializowalne,
- nie jest możliwe przekazywanie z warstwy EJB obiektów encyjnych, których pola są pobierane z opóźnieniem (tzw. lazy fetch):
  - można wymusić stosowanie natychmiastowego pobierania wszystkich pól,
  - można udostępnić zestaw funkcji pozwalających na pobieranie dodatkowych danych z opóźnieniem;
- może być wymagane spakowanie definicji interfejsów EJB i klas encyjnych zarówno z modułem EJB jak i z wykorzystującymi go modułami webowymi lub klienckimi:
  - można wyłączyć te elementy do oddzielnego archiwum JAR.

Tworzenie EJB

Implementacja EJB składa się z:

- klasy implementujące funkcjonalność EJB,
- opcjonalne interfejsy biznesowe:
  - remote - dla dostępu zdalnego do EJB,
  - local - dla dostępu lokalnego do EJB;
- inne klasy, z których korzystają EJB,
- opcjonalne deskryptory opisujące EJB:
  - deskryptor standardowy - `ejb-jar.xml`,
  - deskryptor zależny od serwera, np.: `jboss-ejb3.xml`.

Sesyjne EJB

Klasa beana:

- musi posiadać odpowiednią adnotację (`@Stateless`, `@Statefull`, `@Singleton`),
- musi być publiczna i nie może być abstrakcyjna ani finalna,
- musi implementować metody biznesowe,
- musi mieć publiczny, bezparametrowy konstruktor,
- nie może definiować metody `finalize()`,
- może posiadać metody asynchroniczne (`@Asynchronous`),
- nazwy metod nie mogą zaczynać się od prefiksu `ejb`.

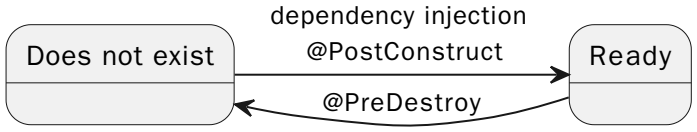
Sesyjne EJB - cyk życia

Możliwość definiowania metod wywoływanych na określonych etapach cyklu życia EJB poprzez odpowiednie adnotacje na metodach o sygnaturze `void ...()`:

- `@PostConstruct` - wywołana po wstrzyknięciu zależności,
- `@PreDestroy` - wywołana przed usunięcie beana przez kontener,
- `@PrePassivate` - wywołana przed „zatrzymaniem” beana stanowego,
- `@PostActivate` - wywołana po aktywowaniu „zatrzymanego” beana stanowego.

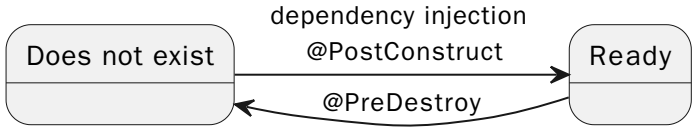
POLITECHNIKA GDAŃSKA

Cykl życia bezstanowego sesyjnego EJB

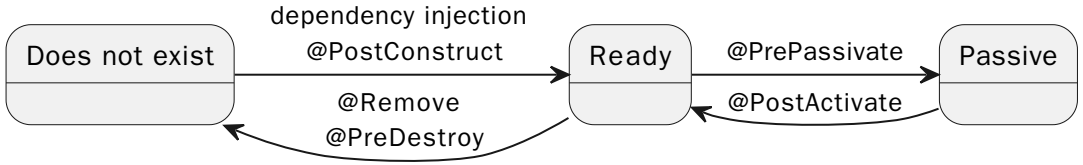


POLITECHNIKA GDAŃSKA

Cykl życia sesyjnego singletona EJB



Cykl życia stanowego sesyjnego EJB



Sesyjne EJB - bez interfjesu

Bezstanowy sesyjny bean EJB niedostępny przez interfejs (no-interface view):

```
@Stateless
public class LibraryService {
    // ...
}
```

```
@ViewScoped
@Named
public class BookView {

    @EJB
    private LibraryService service;

}
```

Sesyjne EJB - interfejs lokalny

Interfejs lokalny:

```
@Local
public interface LibraryLocal {
  // ...
}
```

Bezstanowy sesyjny bean EJB dostępny przez lokalny interfejs:

```
@Stateless
public class LibraryService implements LibraryLocal {
  // ...
}
```

```
@ViewScoped
@Named
public class BookView {

    @EJB
    private LibraryLocal service;

}
```

Sesyjne EJB - interfejs lokalny cd

Interfejs:

```
public interface LibraryLocal {
  // ...
}
```

Bezstanowy sesyjny bean EJB dostępny przez lokalny interfejs:

```
@Local(LibraryLocal.class)
@Stateless
public class LibraryService implements LibraryLocal {
  // ...
}
```

```
@ViewScoped
@Named
public class BookView {

    @EJB
    private LibraryLocal service;

}
```

Sesyjne EJB - bez interfejsu

Bezstanowy sesyjny bean EJB nie dostępny przez interfejs:

```
@LocalBean
@Stateless
public class LibraryService {
    // ...
}
```

```
@ViewScoped
@Named
public class BookView {

    @EJB
    private LibraryService service;

}
```

Sesyjne EJB - interfejs zdalny

Interfejs zdalny:

```
@Remote
public interface LibraryRemote {
    // ...
}
```

Bezstanowy sesyjny bean EJB dostępny przez zdalny interfejs oraz lokalnie bez interfejsu:

```
@Stateless
@LocalBean
public class LibraryService implements LibraryRemote {
    // ...
}
```

```
@ViewScoped
@Named
public class BookView {

    @EJB
    private LibraryService service;

}
```

Sesyjne EJB - interfejs zdalny cd

Interfejs zdalny:

```
@Remote
public interface LibraryRemote {
    // ...
}
```

Bezstanowy sesyjny bean EJB dostępny przez zdalny interfejs:

```
@Stateless
public class LibraryService implements LibraryRemote {
    // ...
}
```

```
@ViewScoped
@Named
public class BookView {

    @EJB(lookup = "java:global/library-ear/library-ejb/LibraryService!pl.LibraryRemote")
    private LibraryRemote service;

}
```

Wyszukiwanie przez JNDI

Gdy wstrzykiwanie nie jest możliwe:

```
LibraryRemote service = InitialContext
        .doLookup("java:global/library-ear/library-ejb/LibraryService!pl.LibraryRemote");
```

Adresy JNDI - Java Naming and Directory Interface - usługa katalogowa pozwalający klientom na wyszukiwanie obiektów za pomocą nazw:

- `java:global` - wyszukiwanie zdalnych beanów:
  - `java:global[/application name]/module name/enterprise bean name[!interface name]`;
- `java:module` - wyszukiwanie beanów w tym samym module:
  - `java:module/enterprise bean name[!interface name]`;
- `java:app` - wyszukiwanie beanów w tej samej aplikacji:
  - `java:app[/module name]/enterprise bean name[!interface name]`.

Sesyjne EJB - singleton

Po prostu singleton:

```
@Singleton
public class BookSingleton {
    // ...
}
```

```
@ViewScoped
@Named
public class BookView {

    @EJB
    private BookSingleton singleton;

}
```

Singleton uruchamiany automatycznie z aplikacją:

```
@Singleton
@Startup
public class InitBooks {

}
```

Sesyjne EJB - singleton - zależności

Zależności pomiędzy singletonami:

```
@Singleton
@Startup
public class InitUsers {

}
```

```
@Singleton
@Startup
@DependsOn("InitUsers")
public class InitBooks {

}
```

Singleton - synchronizacja

Singleton a wątki:

- singleton w EJB został zaprojektowany z myślą o dostępie wielowątkowym,
- dwa tryby zarządzania synchronizacja, ustawiane za pomocą `@ConcurrencyManagement`:
  - kontener (adnotacja `@Lock`),
  - bean (`synchronized`);
- kontener pozwala na dwa rodzaje synchronizacji:
  - `LockType.WRITE` - wykluczający dostęp do metod instancji,
  - `LockType.READ` - współdzielony dostęp do metod instancji;
- domyślnie synchronizacja jest realizowana przez kontener (`@Lock(LockType.WRITE)` na każdej metodzie)).

Singleton - synchronizacja - przykład

Przykład synchronizacji:

```
@Singleton
@ConcurrencyManagement(ConcurrencyManagementType.CONTAINER)
public class LibrarySingleton {

    @Lock(LockType.WRITE)
    public void allowSingle() {

    }

    @Lock(LockType.READ)
    public void allowMany() {

    }

}
```

Pomocne źródła

Specyfikacja:

- https://jakarta.ee/specifications/enterprise-beans/4.0/jakarta-enterprise-beans-spec-core-4.0

Uzupełnienie informacji:

- https://eclipse-ee4j.github.io/jakartaee-tutorial/#enterprise-beans
- https://www.baeldung.com/ejb-intro
- https://www.baeldung.com/wildfly-ejb-jndi
- https://www.baeldung.com/ejb-session-beans